

Kubernetes Storage Deep Dive

Volumes / PersistentVolume (PV) / PersistentVolumeClaim (PVC) / StorageClass

1 Kubernetes Volumes

What is a Volume in Kubernetes?

In Kubernetes (K8s), a **Volume** is a directory accessible to containers in a Pod. Unlike container filesystems (which are ephemeral), volumes preserve data beyond container restarts.

- ◆ Volumes are defined at the **Pod level**
 - ◆ Containers in the same Pod can share volumes
 - ◆ Volume lifecycle depends on the type used
-

Why Volumes Are Needed

Containers are ephemeral. When a container crashes or restarts:

- Its writable layer is deleted
- Data inside the container is lost

Volumes solve this by:

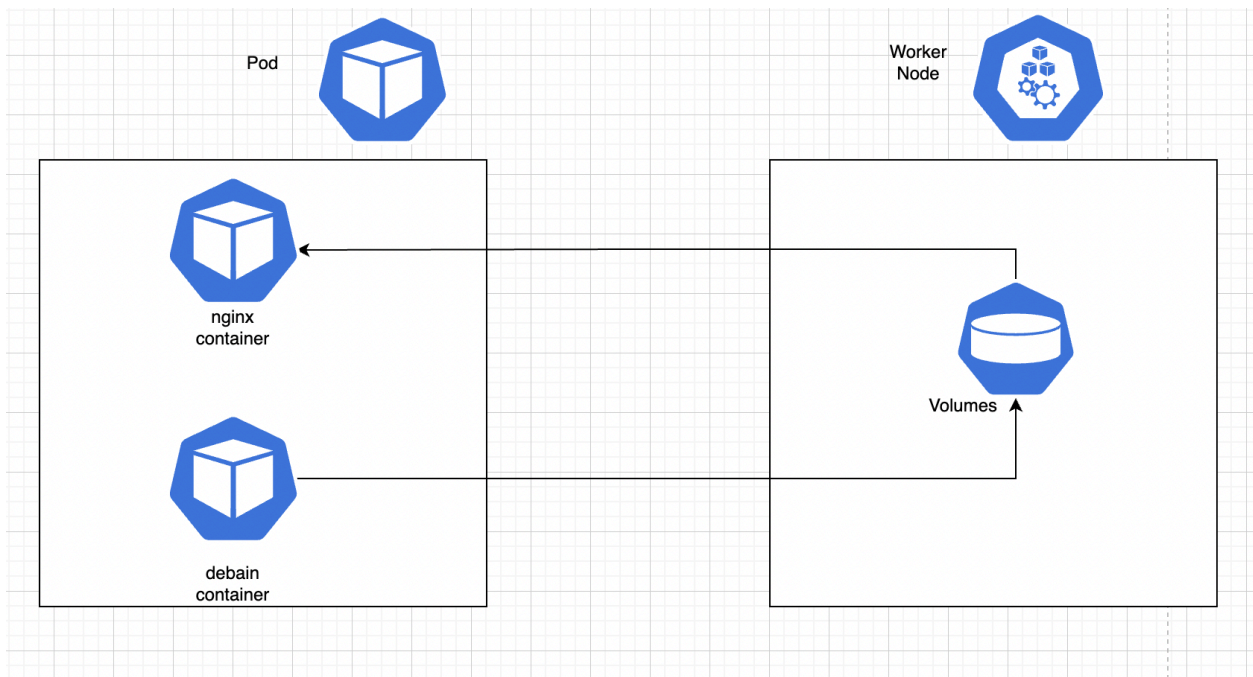
- Persisting data
 - Sharing data between containers
 - Mounting configuration or secrets
 - Connecting to external storage systems
-

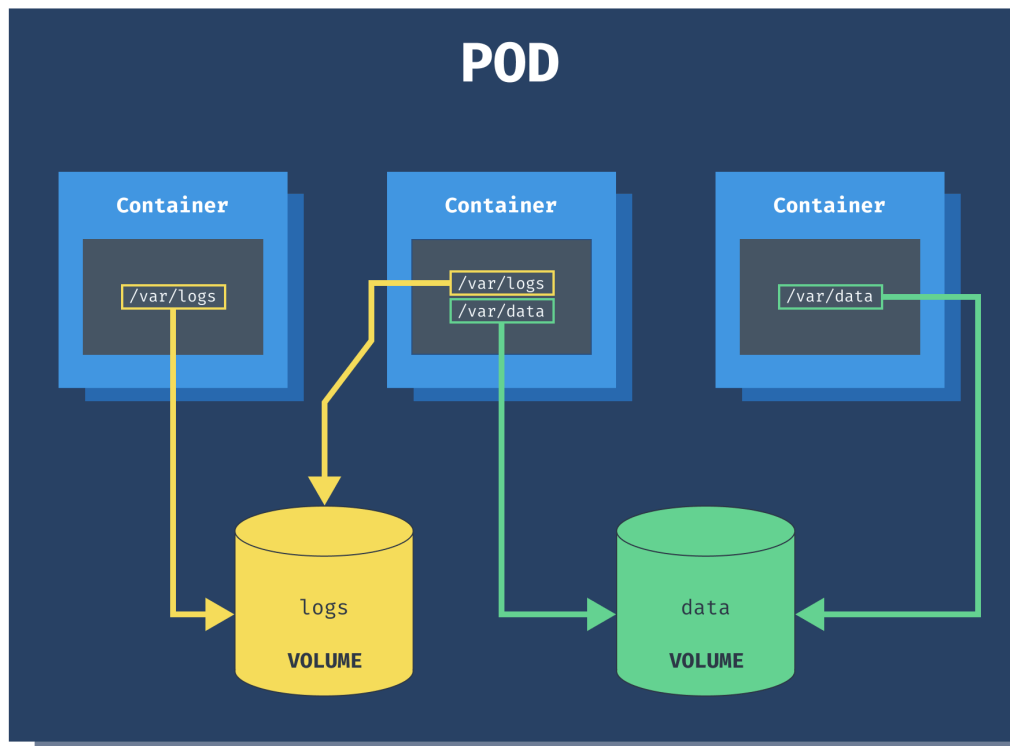
Volume Lifecycle

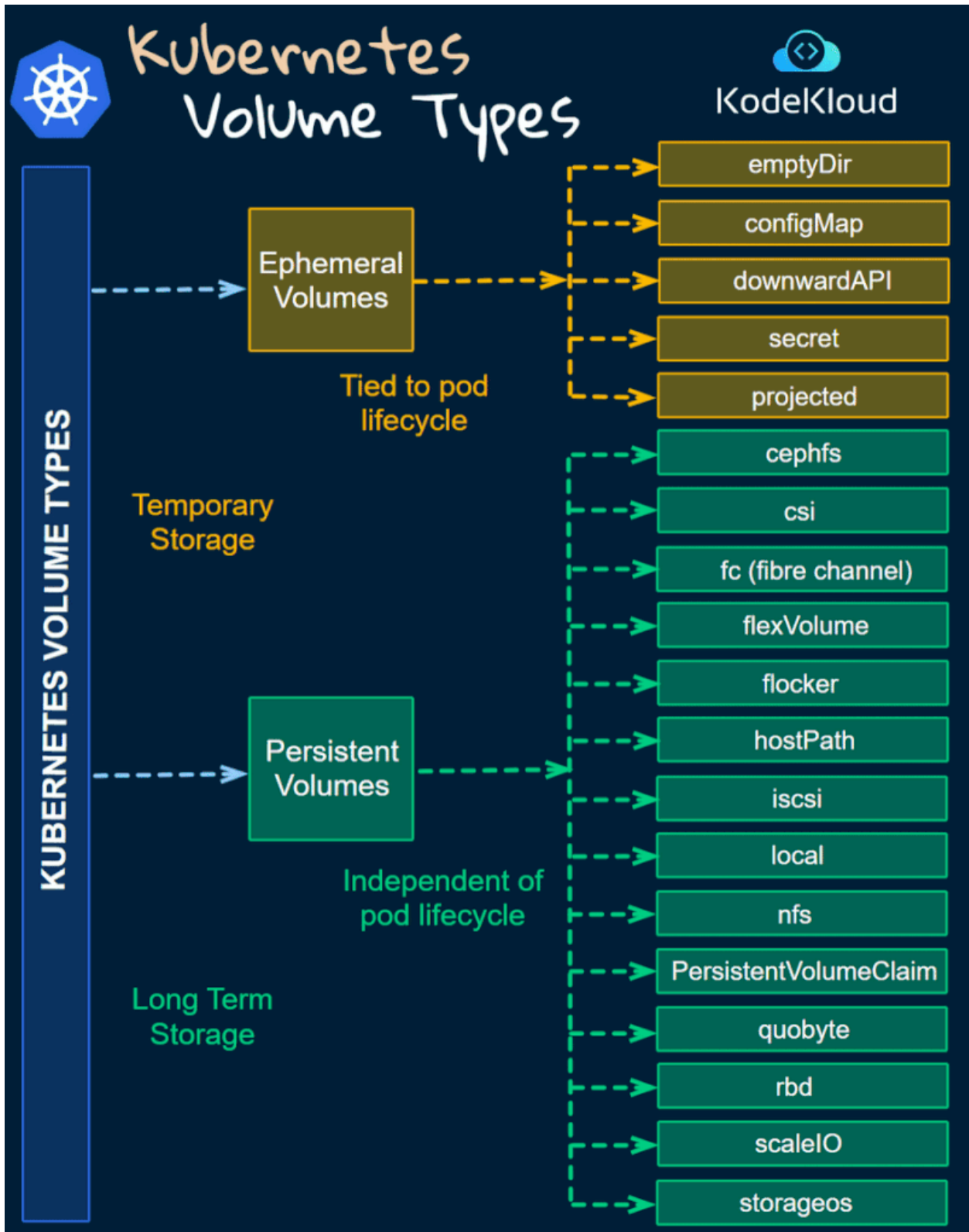
Scope	Lifecycle
Container	Short-lived
Pod	Short-lived
Volume	Depends on type (can outlive Pods)

Types of Kubernetes Volumes

1 emptyDir







- Created when Pod starts

- Deleted when Pod is removed
- Used for:
 - Scratch space
 - Cache
 - Sharing data between containers in same Pod

Example:

apiVersion: v1

kind: Pod

spec:

containers:

- name: app

image: nginx

volumeMounts:

- mountPath: /cache

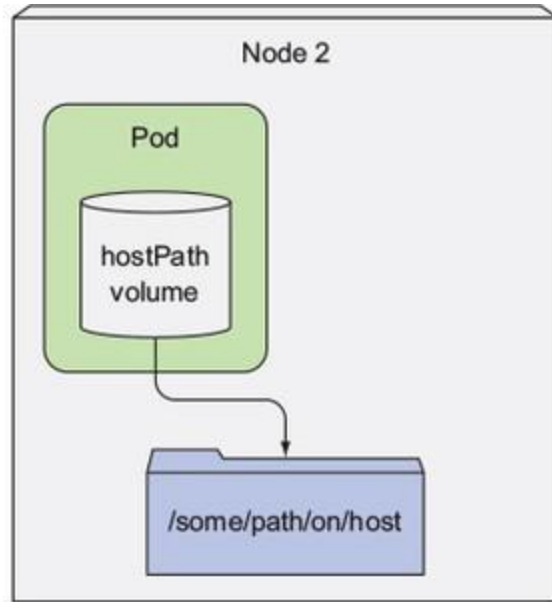
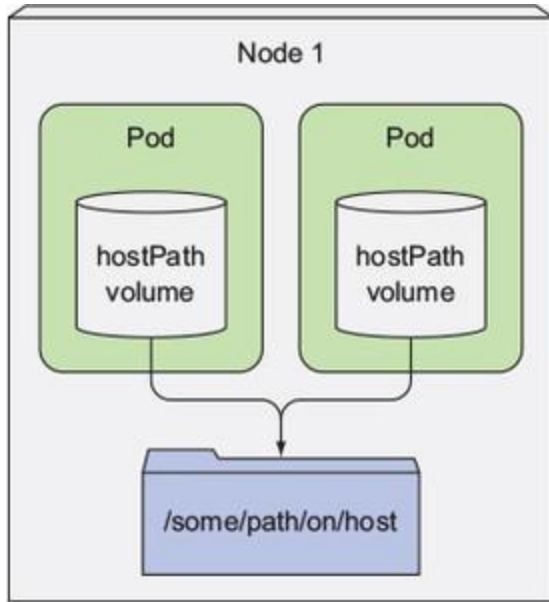
name: cache-volume

volumes:

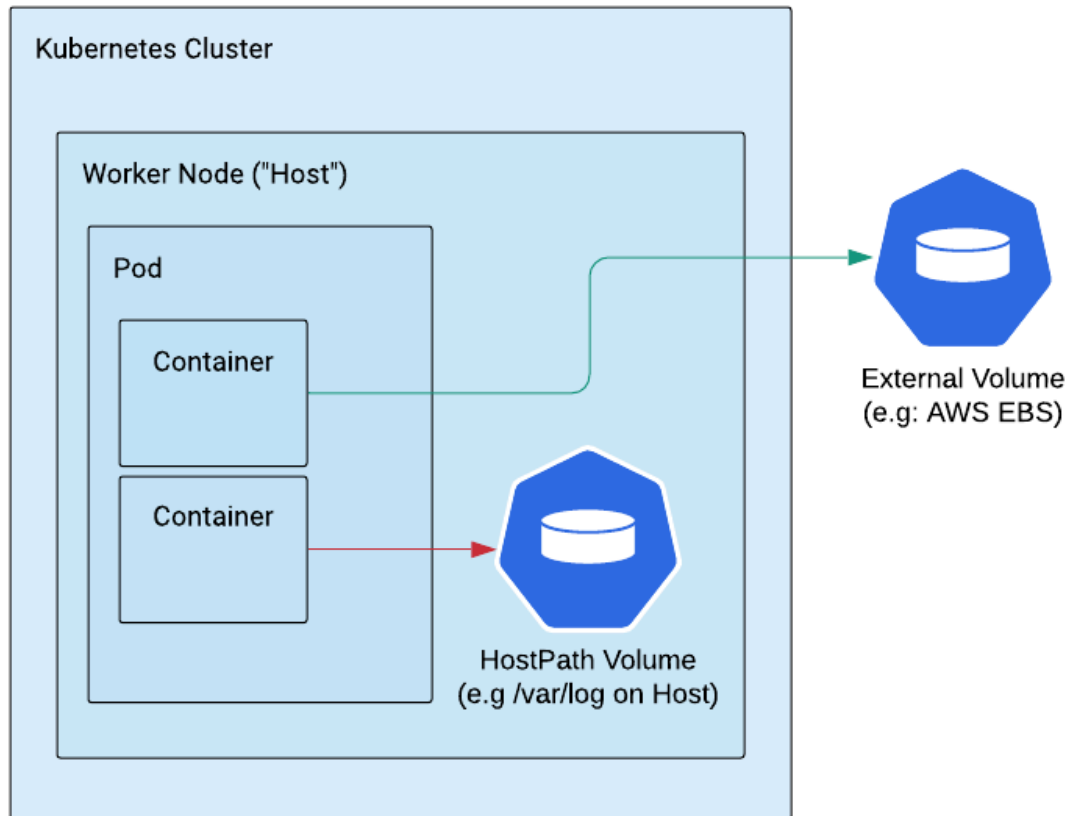
- name: cache-volume

emptyDir: {}

2 hostPath

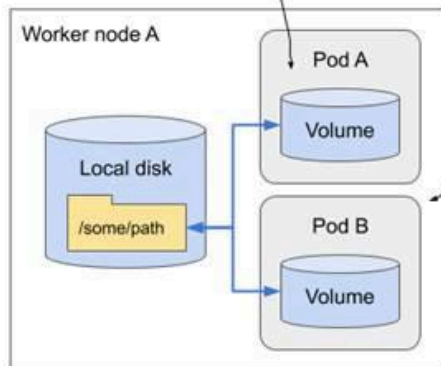


HostPath vs External Volume

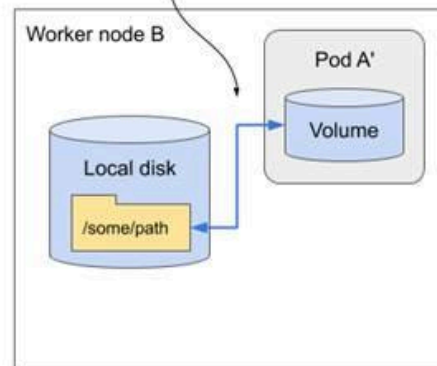


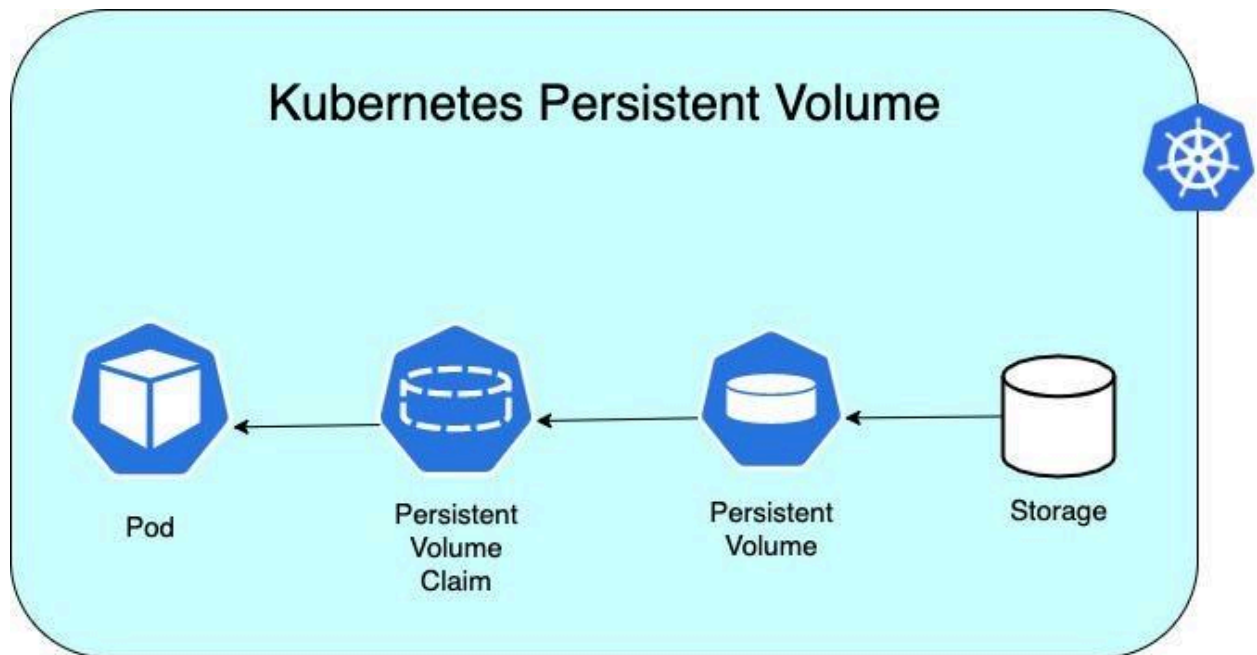
A hostPath volume maps to a file or directory on the node's local filesystem.

Pods on the same node can share data by using the same path in their hostPath volumes.



If a pod is rescheduled to a different node, the hostPath volume will contain a different set of files.





- Mounts file/directory from node's filesystem
 - Not recommended for production (security risk)
 - Useful for testing or single-node setups
-

3 ConfigMap & Secret Volumes

Used to inject:

- Configuration files
 - Sensitive data (passwords, certificates)
-

4 Network & Cloud Volumes

Examples:

- NFS
- iSCSI
- AWS EBS
- Azure Disk

- GCE Persistent Disk

These enable persistent storage across Pods.

2 PersistentVolume (PV)

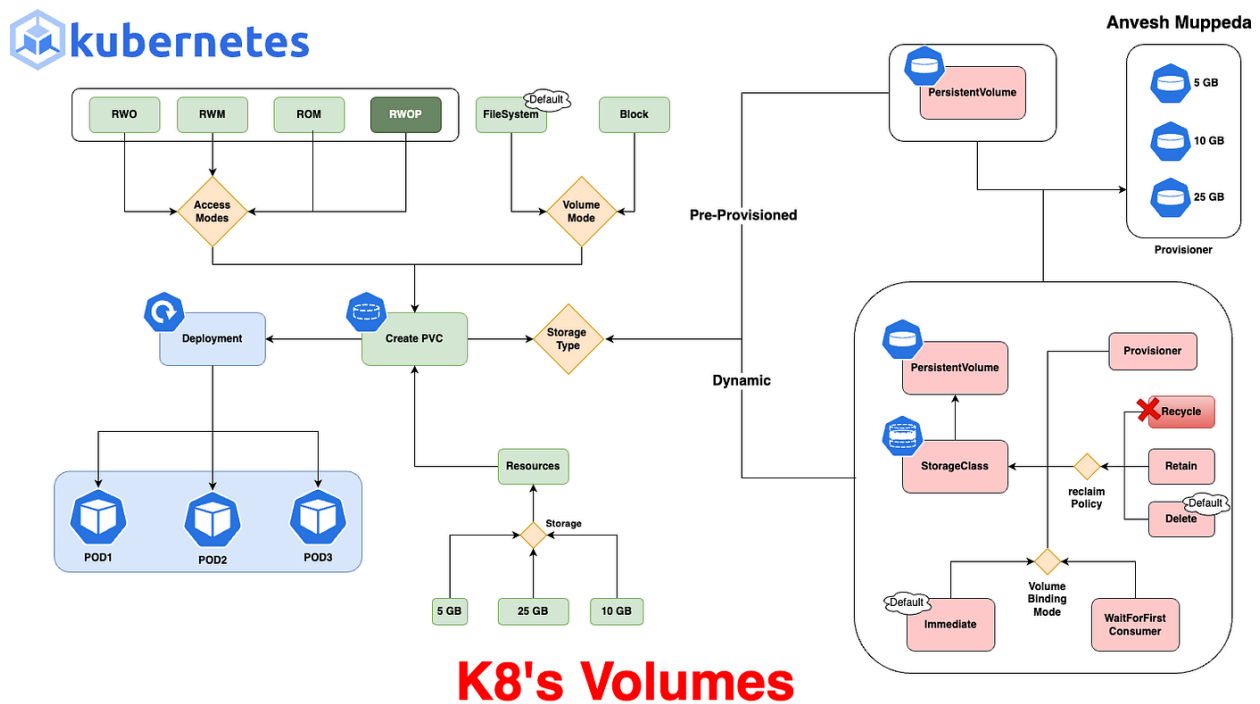
What is a PersistentVolume?

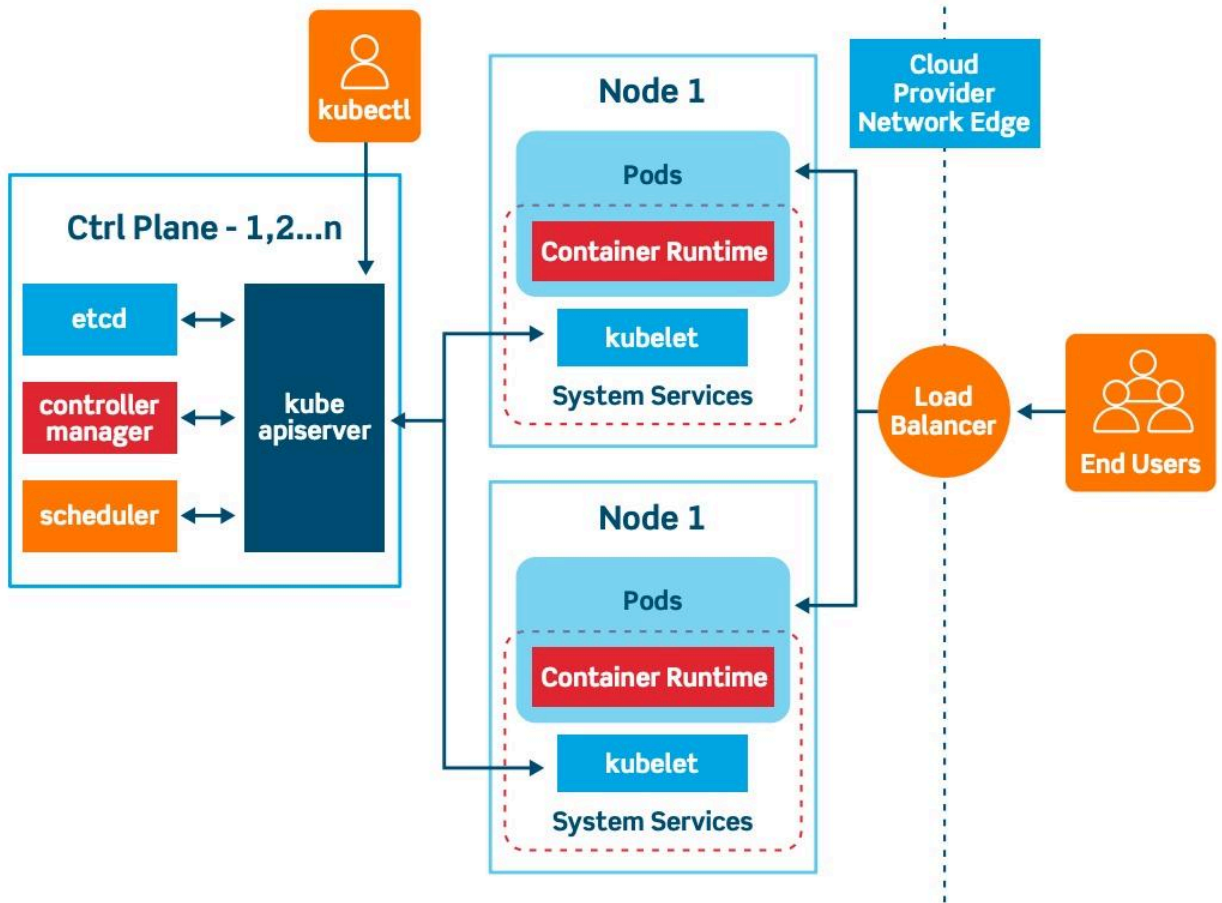
A **PersistentVolume (PV)** is a cluster-wide storage resource provisioned by:

- Administrator (static provisioning)
- Automatically via StorageClass (dynamic provisioning)

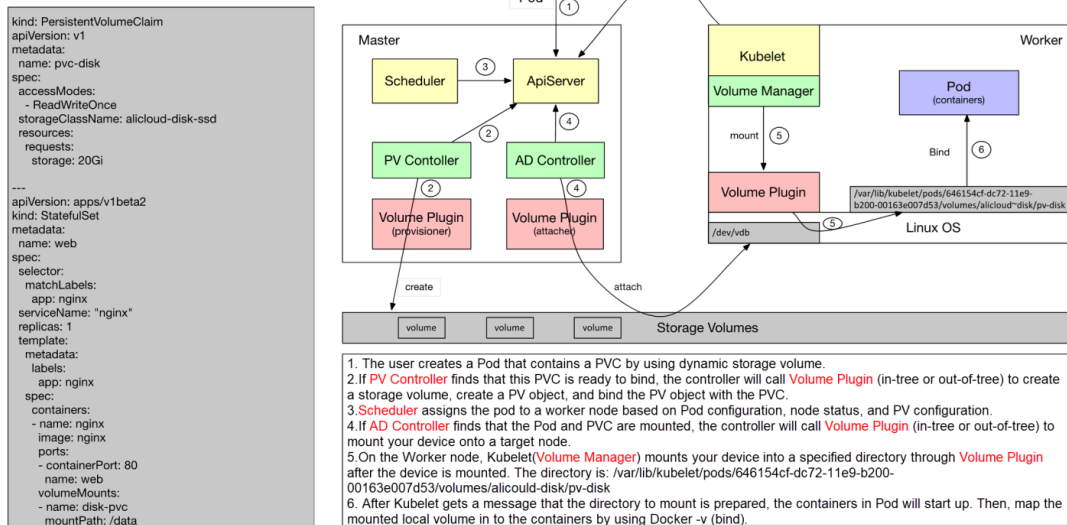
PV is independent of Pod lifecycle.

PV Architecture





Process of Mounting a Volume in Kubernetes



Key PV Properties

apiVersion: v1

kind: PersistentVolume

spec:

capacity:

storage: 10Gi

accessModes:

- ReadWriteOnce

persistentVolumeReclaimPolicy: Retain

storageClassName: standard

Important Fields

Field

Meaning

capacity

Size of storage

accessModes	How volume can be mounted
reclaimPolicy	What happens after PVC deletion
storageClassName	Associated StorageClass

Access Modes

Mode	Meaning
ReadWriteOnce (RWO)	Mounted by one node
ReadOnlyMany (ROX)	Read-only by many nodes
ReadWriteMany (RWX)	Read-write by many nodes

Reclaim Policies

Policy	Behavior
Retain	Keeps data after PVC deletion
Delete	Deletes underlying storage
Recycle (deprecated)	Basic scrub (removed in modern K8s)

3 PersistentVolumeClaim (PVC)

What is PVC?

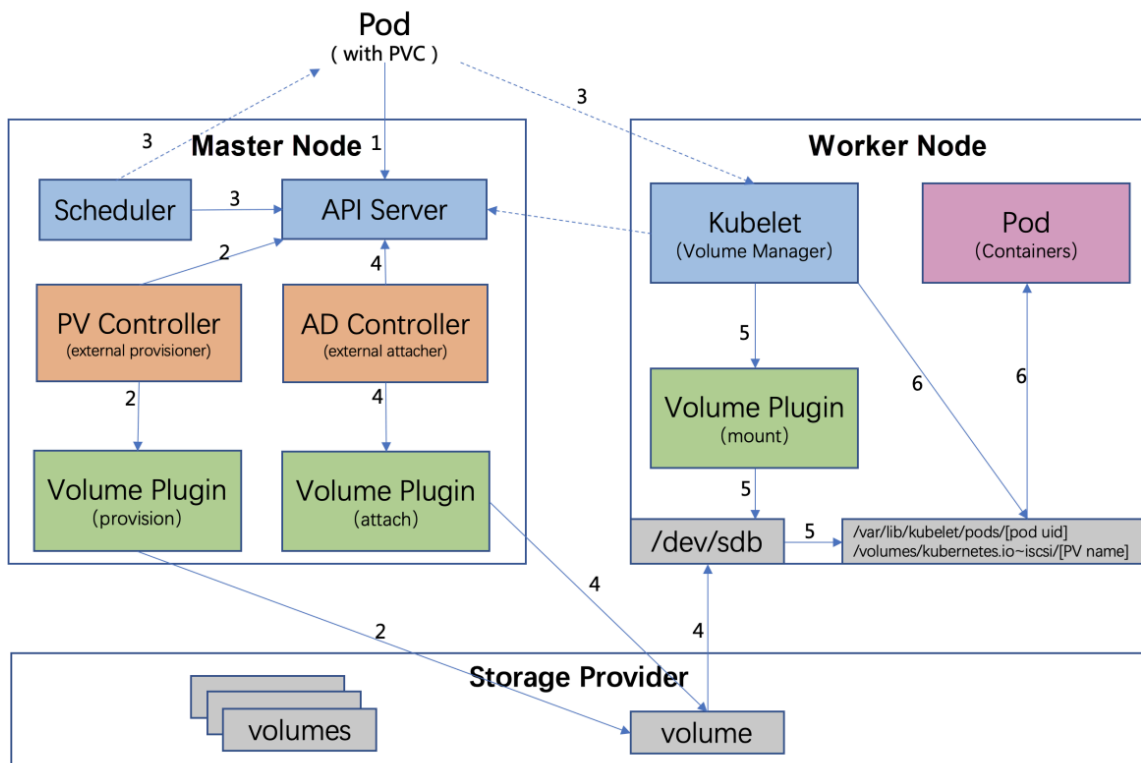
A **PersistentVolumeClaim (PVC)** is a request for storage by a user.

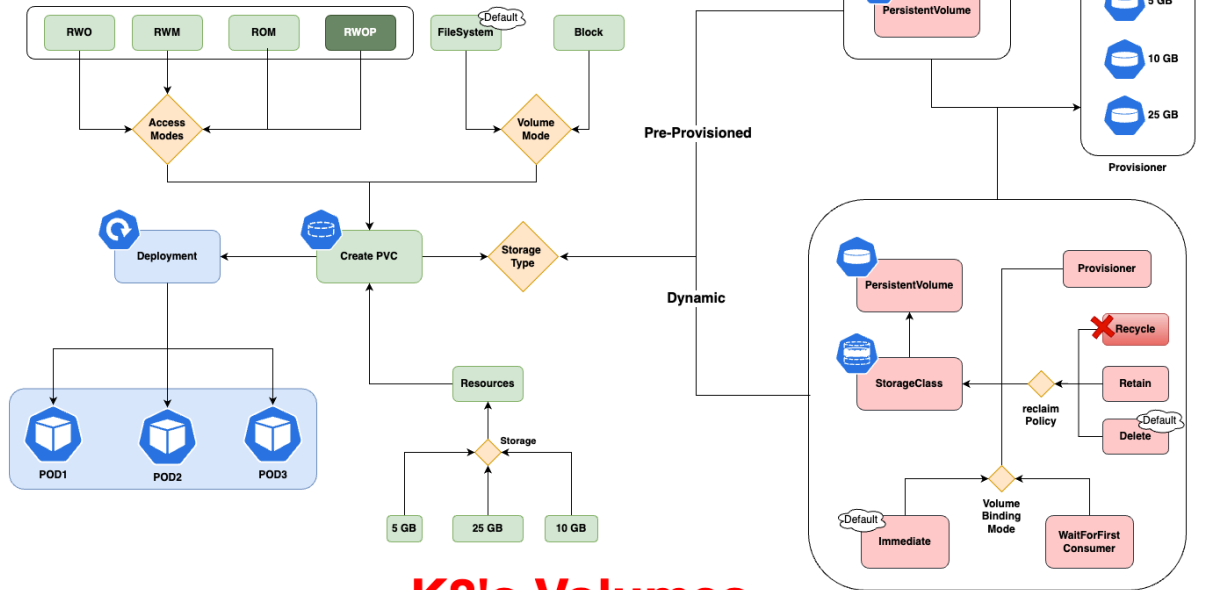
Think of it as:

 "I need 5Gi storage with ReadWriteOnce"

Kubernetes binds PVC to a matching PV.

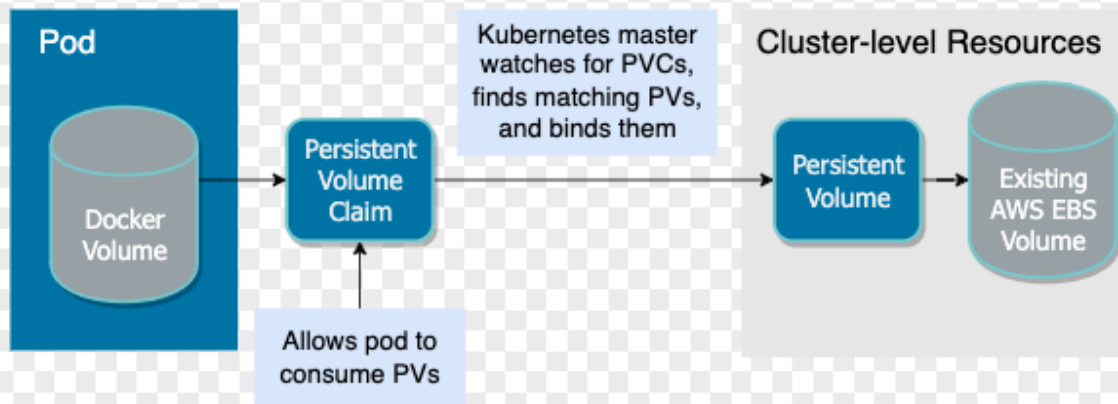
PVC Flow



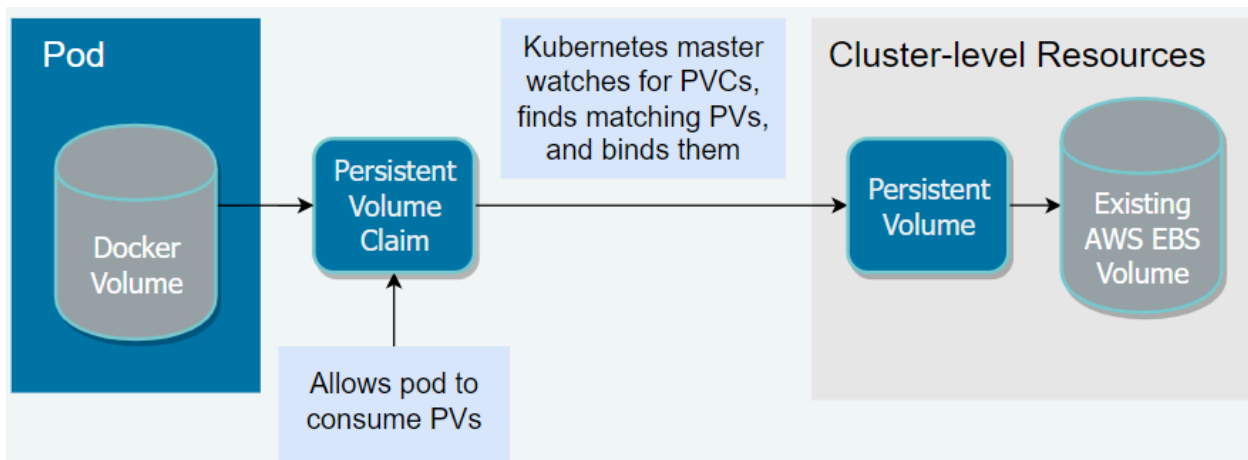
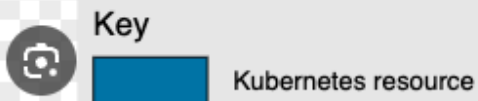
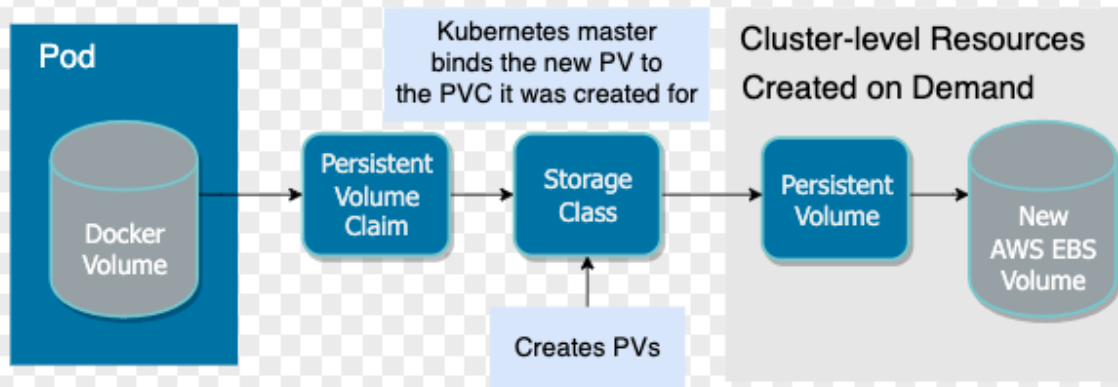


K8's Volumes

Setting Up Existing Persistent Volumes



Dynamically Provisioning New Persistent Volumes



PVC Example

apiVersion: v1

kind: PersistentVolumeClaim

spec:

accessModes:

- ReadWriteOnce

resources:

requests:

storage: 5Gi

storageClassName: standard

Binding Process

1. User creates PVC
 2. Kubernetes searches for matching PV
 3. If found → Bound
 4. If not found → Wait (or dynamically provision)
-

4 StorageClass

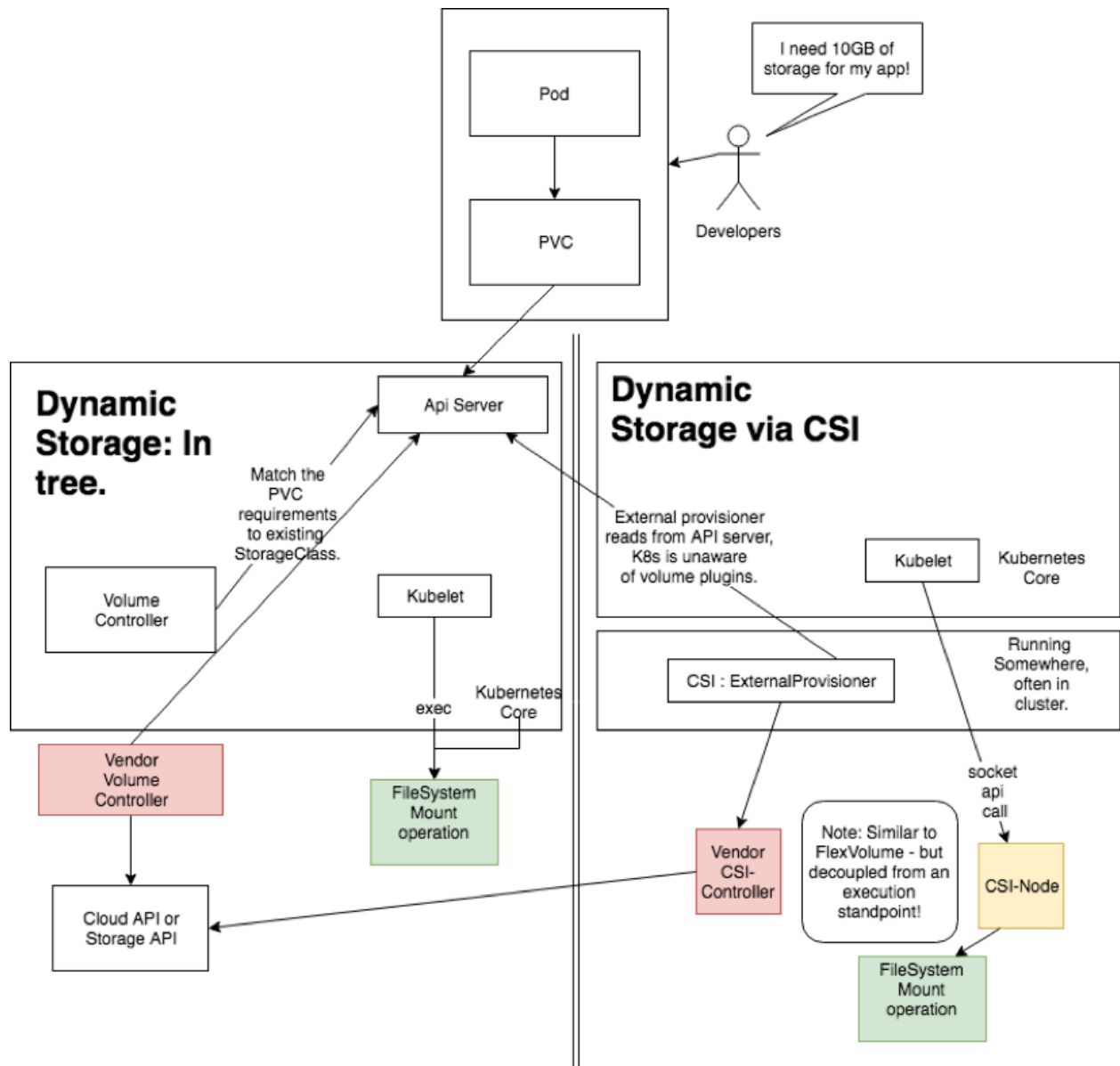
What is StorageClass?

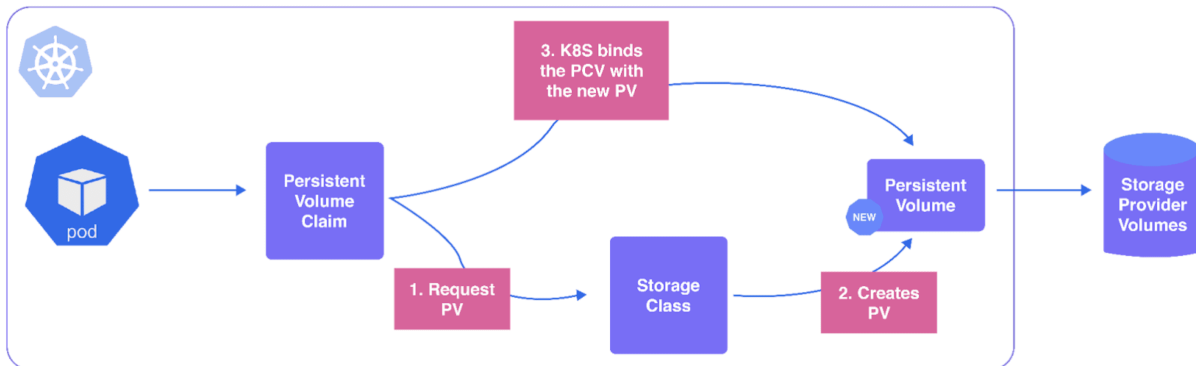
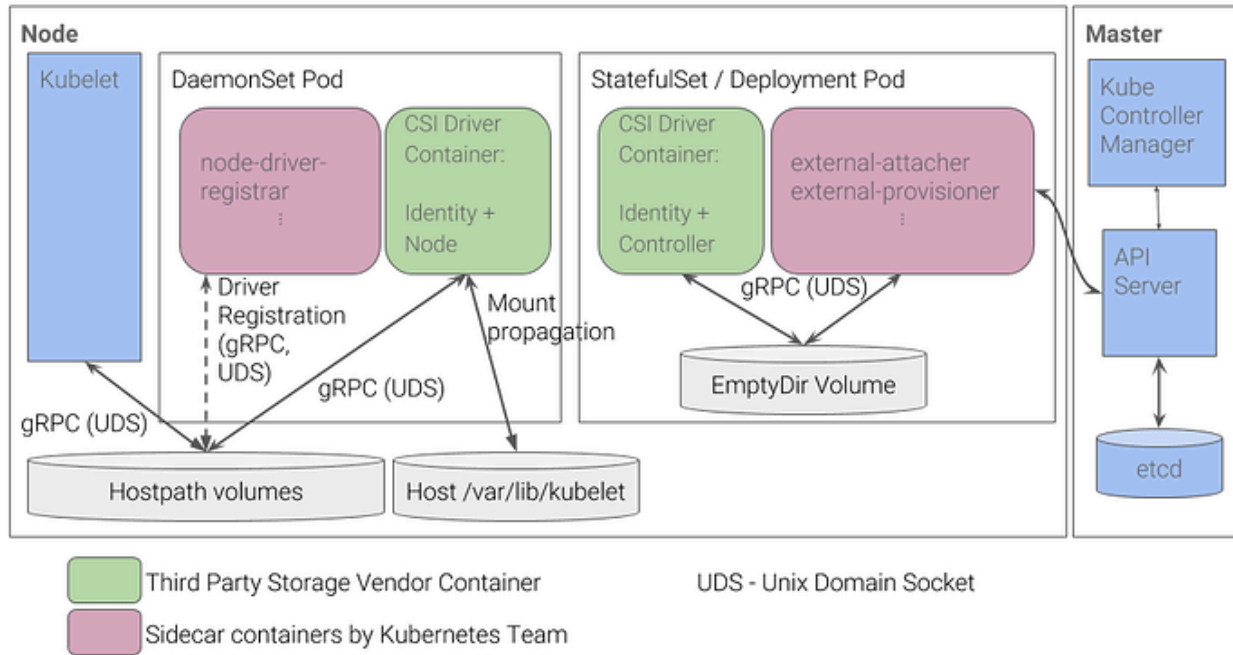
A **StorageClass** enables dynamic provisioning of storage.

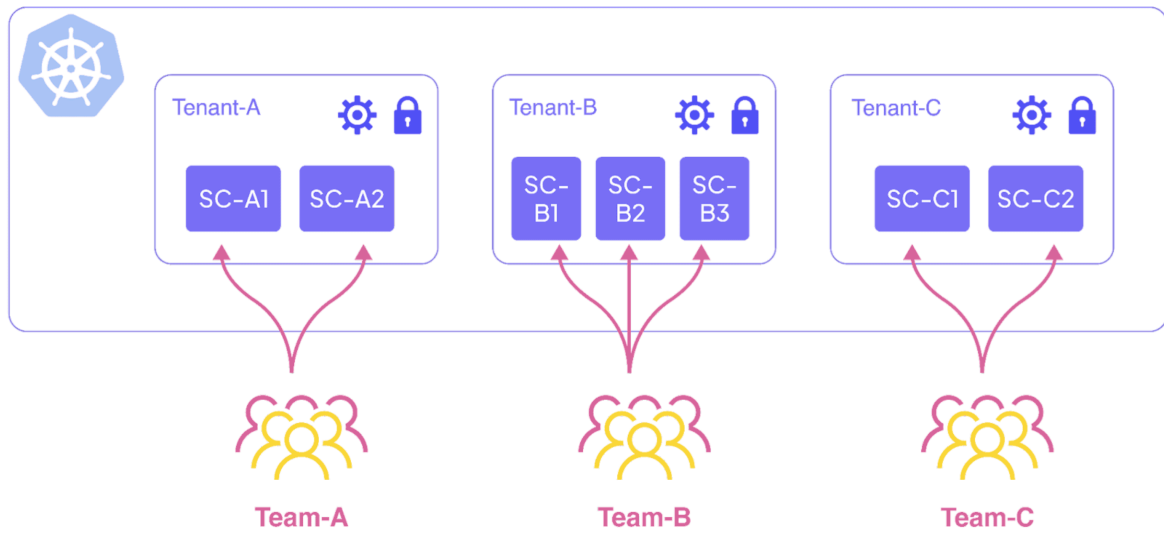
Instead of manually creating PVs:

- PVC requests storage
 - StorageClass automatically creates PV
-

StorageClass Architecture







StorageClass Example

apiVersion: storage.k8s.io/v1

kind: StorageClass

metadata:

name: standard

provisioner: kubernetes.io/aws-ebs

parameters:

type: gp3

reclaimPolicy: Delete

volumeBindingMode: WaitForFirstConsumer

Important StorageClass Fields

Field	Meaning
provisioner	CSI driver
parameters	Storage backend config

reclaimPolicy	What happens when PVC deleted
---------------	-------------------------------

volumeBindingMode	When binding occurs
-------------------	---------------------

volumeBindingMode Options

Mode	Behavior
Immediate	Provision instantly
WaitForFirstConsumer	Wait until Pod is scheduled (better for topology-aware storage)

5 Static vs Dynamic Provisioning

Static Provisioning

Admin creates PV manually → PVC binds to it.

Pros:

- More control
- Predictable

Cons:

- Operational overhead

Dynamic Provisioning

PVC + StorageClass → PV created automatically.

Pros:

- Scalable
- Cloud-native
- Automated

Cons:

- Less manual control
-

6 End-to-End Example

Step 1 – Create StorageClass

kind: StorageClass

metadata:

name: fast

provisioner: kubernetes.io/aws-ebs

Step 2 – Create PVC

kind: PersistentVolumeClaim

metadata:

name: app-storage

spec:

storageClassName: fast

accessModes:

- ReadWriteOnce

resources:

requests:

storage: 10Gi

Step 3 – Use PVC in Pod

```
apiVersion: v1
kind: Pod
spec:
  containers:
  - name: app
    image: nginx
    volumeMounts:
    - mountPath: /data
      name: storage
  volumes:
  - name: storage
    persistentVolumeClaim:
      claimName: app-storage
```

7 Real-World Production Considerations

1 Backup Strategy

Use:

- Velero
- Snapshot APIs
- Cloud provider snapshots

2 Stateful Applications

Use:

- StatefulSets
- VolumeClaimTemplates

3 Storage Performance

Consider:

- IOPS
- Throughput
- Latency
- SSD vs HDD

4 Security

- Encrypt volumes
- Use RBAC
- Avoid hostPath in production

8 Comparison Summary

Feature	Volume	PV	PVC	StorageClass
Scope	Pod	Cluster	Namespace	Cluster
Created By	Developer	Admin / SC	Developer	Admin
Lifecycle	Pod	Independent	Independent	Independent
Purpose	Mount storage	Provide storage	Request storage	Auto-provision

9 Interview-Level Summary

- Volumes solve container ephemerality.
- PV is the actual storage resource.
- PVC is the request.
- StorageClass enables dynamic provisioning.
- AccessModes control mount behavior.
- ReclaimPolicy controls data lifecycle.
- volumeBindingMode optimizes scheduling.

10 Final Architecture View

Pod

└ PVC

└ StorageClass

└ PV

└ Physical Storage (EBS / NFS / etc.)